CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 02
Duration: 25 Minutes

A

Name:	ID:	Section:

Question 1 [CO1] [10 Points]

A robotics lab is testing **N mini robots over R rounds**. In each round, each robot sends a **two-digit signal code**. From each **valid code**, the **bigger digit** is the **power level,** and the **smaller digit** is the **stability level**. If the **difference between them is 5 or more**, the robot is **unstable**. Otherwise, it is **stable**. You are required to write a Java program that **takes R and N as input,** followed by the signal code of each robot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit** is **divisible by 3** and the **smaller digit** is **divisible by 7,** the code is **invalid,** and **input must be taken again** for that robot in that round. After each round, print the number of **stable and unstable robots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

Sample Input	Sample Output	Explanation
Rounds (R) = 2 Robots (N) = 3 Round 1: Robot 1 code = 42 Robot 2 code = 97 Robot 2 code = 81 Robot 3 code = 65 Round 2: Robot 1 code = 54 Robot 2 code = 20 Robot 3 code = 31	Round 1: Stable robots: 2 Unstable robots: 1 Round 2: Stable robots: 3 Unstable robots: 0	Round 1: 42 → bigger = 4, smaller = 2 → difference = 2 → Stable 97 → bigger = 9, smaller = 7 → bigger divisible by 3 and smaller divisible by 7 → Invalid 81 → taken again for robot 2 → bigger = 8, smaller = 1 → difference = 7 → Unstable 65 → bigger = 6, smaller = 5 → difference = 1 → Stable So, in Round 1: Stable robots = 2 Unstable robots = 1

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 02
Duration: 25 Minutes

B

Name:	ID:	Section:

Question 1 [CO1] [10 Points]

A game company is testing **N mini game bots over R rounds**. In each round, each bot sends a **two-digit performance code**. From each **valid code**, the **bigger digit** is the **attack level**, and the **smaller digit** is the **defense level**. If the **difference** between them is **less than 4**, the bot is **balanced**. Otherwise, it is **imbalanced**. You are required to write a Java program that takes **R and N as input**, followed by **the performance code** of each bot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit. If the bigger digit is divisible by 5 and the smaller digit is divisible by 3,** the code is **invalid, and input must be taken again** for that bot in that round. After each round, print the number of **balanced and imbalanced bots.**

[Assume all inputs are valid two-digit integers from 10 to 99.]

Sample Input	Sample Output	Explanation
Rounds (R)= 2 Bots (N)= 3 Round 1: Bot 1 code = 39 Bot 2 code = 11 Bot 3 Code = 81 Round 2: Bot 1 code = 42 Bot 2 code = 53 Bot 2 code = 87 Bot 3 Code = 64	Round 1: Balanced bots: 1 Imbalanced bots: 2 Round 2: Balanced bots: 3 Imbalanced bots: 0	Round 2: 42 → bigger = 4, smaller = 2 → difference = 2 → Balanced 53 → bigger = 5, smaller = 3 → bigger is divisible by 5 and smaller is divisible by 3 → Invalid 87 → taken again for bot 2 → bigger = 8, smaller = 7 → difference = 1 → Balanced 64 → bigger = 6, smaller = 4 → difference = 2 → Balanced So, in Round 2: Balanced bots = 3 Imbalanced bots = 0

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class setA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Rounds (R)= ");
        int R = sc.nextInt();
        System.out.print("Robots (N)= ");
        int N = sc.nextInt();

        for (int round = 1; round <= R; round++) {
            int stable = 0;
            int unstable = 0;

            System.out.println("Round " + round + ":");
            for (int robot = 1; robot <= N; robot++) {
                System.out.print("Robot "+robot+" code = ");
                int code = sc.nextInt();

                int d1 = code / 10;
                int d2 = code % 10;

                int bigger, smaller;
                if (d1 > d2) {
                    bigger = d1;
                    smaller = d2;
                } else {
                    bigger = d2;
                    smaller = d1;
                }

                if (bigger % 3 == 0 && smaller % 7 == 0) {
                    robot--;   // take input again for same robot
                    continue;
                }

                int diff = bigger - smaller;

                if (diff >= 5) {
                    unstable++;
                } else {
                    stable++;
                }
            }
            System.out.println("Round " + round + ":");
            System.out.println("Stable robots: " + stable);
            System.out.println("Unstable robots: " + unstable);
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Takes R and N as input correctly	1
2	Uses 2 nested loops correctly for rounds and robots	2
3	Extracts the two digits correctly	1
4	Finds bigger and smaller digits correctly	1
5	Checks invalid condition correctly (bigger % 3 == 0 && smaller % 7 == 0)	1
6	Retakes input for the same robot in the same round	1
7	Checks balanced/imbalanced condition correctly (difference >= 5)	1
8	Counts stable and unstable robots correctly for each round	1
9	Prints the correct round-wise output	1

SET - [B]

SOLUTION

```
import java.util.Scanner;

public class SetB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Rounds (R) = ");
        int R = sc.nextInt();
        System.out.print("Bots (B) = ");
        int N = sc.nextInt();

        for (int round = 1; round <= R; round++) {
            int balanced = 0;
            int imbalanced = 0;

            System.out.println("Round "+round+":");
            for (int bot = 1; bot <= N; bot++) {
                System.out.print("Bot "+bot+" code = ");
                int code = sc.nextInt();

                int d1 = code / 10;
                int d2 = code % 10;

                int bigger, smaller;
                if (d1 > d2) {
                    bigger = d1;
                    smaller = d2;
                } else {
                    bigger = d2;
                    smaller = d1;
                }

                if (bigger % 5 == 0 && smaller % 3 == 0) {
                    bot--;
                    continue;
                }

                int diff = bigger - smaller;

                if (diff < 4) {
                    balanced++;
                } else {
                    imbalanced++;
                }
            }
            System.out.println("Round " + round + ":");
            System.out.println("Balanced bots: " + balanced);
            System.out.println("Imbalanced bots: " + imbalanced);
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Takes R and N as input correctly	1
2	Uses 2 nested loops correctly for rounds and bots	2
3	Extracts the two digits correctly	1
4	Finds bigger and smaller digits correctly	1
5	Checks invalid condition correctly (bigger % 5 == 0 && smaller % 3 == 0)	1
6	Retakes input for the same bot in the same round	1
7	Checks balanced/imbalanced condition correctly (difference < 4)	1
8	Counts balanced and imbalanced bots correctly for each round	1
9	Prints the correct round-wise output	1

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 03
Duration: 25 Minutes

A

Name:	ID:	Section:

Question 1 [CO2] [10 Points]

A cybersecurity team is checking encrypted message logs to identify hidden access codes. A valid code **starts** with **an uppercase letter (A–Z)** and **ends** with **an odd digit (0–9)**, including all characters in between. Once a valid code is found, **it is separated by #**, and the search for the next valid code must continue from the next character. Write a Java program that takes the encrypted message as input from the user, extracts all valid codes according to the rules, and displays them. If no valid code is found, "No valid code found." is displayed.

You are only allowed to use .charAt() and .length() as built-in methods.

Sample Input	Sample Output
abAxy3mnBz2pqC1	Axy3#C1 **Explanation** Axy3 → starts with A, ends with an odd digit 3 (valid) Bz2 → starts with B, ends with an even digit 2 (invalid) C1 → starts with C, ends with an odd digit 1 (valid) Both valid codes are separated by #.
helloXabcYz	No valid code found.

SET-B

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 03
Duration: 25 Minutes

B

Name:	ID:	Section:

Question 1 [CO2] [10 Points]

A research team is scanning DNA samples to identify hidden sample labels. A valid label **starts** with **an even digit (0-9)** and **ends** with a **lowercase letter (a–z)**, including all characters in between. Once a valid label is found, **it is separated by @**, and the search for the next valid code must continue from the next character. Write a Java program that takes a DNA sequence as input from the user, extracts all valid labels according to the rules, and displays them. If no valid label is found, "No valid label found." is displayed.

You are only allowed to use .charAt() and .length() as built-in methods.

Sample Input	Sample Output
2ADxBc5mnDe6qfs	2ADx@6q **Explanation** 2ADx → starts with an even digit 2, ends with x (valid) 5m → starts with an odd digit 5, ends with m (invalid) 6q → starts with an even digit 6, ends with q (valid) Both valid codes are separated by @.
135BCD79EFG	No valid label found.

Solution & Rubric

CSE110 Spring 2026 Lab Exam - 03 Tentative Solutions and Rubrics

SET - [A]

SOLUTION

```java
import java.util.Scanner;
public class LabExam3A {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String message = sc.nextLine();
        String result = "";
        int start = -1;
        for (int i = 0; i < message.length(); i++) {
            char ch = message.charAt(i);
            if (ch >= 'A' && ch <= 'Z') {
                start = i;
            }
            if (start != -1 && (ch=='1'||ch=='3'||ch=='5'||ch=='7'||ch=='9')) {
                String code = "";
                for (int k = start; k <= i; k++) {
                    code = code + message.charAt(k);
                }
                if (result.length() > 0) {
                    result = result + "#";
                }
                result = result + code;
                start = -1;
            }
        }
        if (result.length() == 0) {
            System.out.println("No valid code found.");
        } else {
            System.out.println(result);
        }
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Correctly taking user input using Scanner	1
2	Identifying the start (uppercase letter) and end (odd digit) of a valid code	2.5
3	Correctly extracting each valid code	3
4	Appending # to separate each valid code	2
5	Appropriate print statements for valid code and “No valid code found.”	1.5

SET - [B]

SOLUTION

```java
import java.util.Scanner;
public class DNALabelExtractor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sequence = sc.nextLine();
        String result = "";
        int start = -1;
        for (int i = 0; i < sequence.length(); i++) {
            char ch = sequence.charAt(i);
            if (ch == '0' || ch == '2' || ch == '4' || ch == '6' || ch == '8') {
                start = i;
            }
            if (start != -1 && ch >= 'a' && ch <= 'z') {
                String label = "";
                for (int k = start; k <= i; k++) {
                    label = label + sequence.charAt(k);
                }
                if (result.length() > 0) {
                    result = result + "@";
                }
                result = result + label;
                start = -1;
            }
        }
        if (result.length() == 0) {
            System.out.println("No valid label found.");
        } else {
            System.out.println(result);
        }
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Correctly taking user input using Scanner	1
2	Identifying the start (even digit) and end (lowercase letter) of a valid label	2.5
3	Correctly extracting each valid label	3
4	Appending @ to separate each valid label	2
5	Appropriate print statements for valid label and “No valid label found.”	1.5

BRAC UNIVERSITY
Inspiring Excellence

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 04
Duration: 25 Minutes

A

Name:	ID:	Section:

Question 1 [CO3] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **ascending order** and elements at **odd indices** are in **descending order**. Finally, it displays the count of unique elements and the sorted array.

Given	Expected Output
`int [] marks = {9, -5, 7, 9, -5, 5};`	`Unique count: 4` `Sorted array: 7 5 9 -5`
Explanation: The unique elements are 9, -5, 7, and 5, so the unique count is 4. Even indices are sorted in ascending order: 7, 9 Odd indices are sorted in descending order: 5, -5	

SET-B

BRAC UNIVERSITY
Inspiring Excellence

CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 04
Duration: 25 Minutes

B

Name:	ID:	Section:

Question 1 [CO3] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **descending order** and elements at **odd indices** are in **ascending order**. Finally, it displays the count of unique elements and the sorted array.

Given	Expected Output
`int [] scores= {4, 2, 6, 1, 4, 1};`	`Unique count: 4` `Sorted array: 6 1 4 2`
Explanation: The unique elements are 4, 2, 6, and 1, so the unique count is 4. Even indices are sorted in descending order: 6, 4 Odd indices are sorted in ascending order: 1, 2	

Solution & Rubric

CSE110 Spring 2026 Lab Exam - 04 Tentative Solutions and Rubrics

SET - [A]

SOLUTION

```
public class SetA {
    public static void main(String[] args) {

        int[] marks = {9, -5, 7, 9, -5, 5};
        int[] unique = new int[marks.length];
        int size = 0;

        for (int i = 0; i < marks.length; i++) {
            boolean isDuplicate = false;
            for (int j = 0; j < size; j++) {
                if (marks[i] == unique[j]) {
                    isDuplicate = true;
                    break;
                }
            }
            if (isDuplicate==false) {
                unique[size] = marks[i];
                size++;
            }
        }

        for (int i = 0; i < size; i += 2) {
            int minIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] < unique[minIndex]) {
                    minIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[minIndex];
            unique[minIndex] = tempVal;
        }
        for (int i = 1; i < size; i += 2) {
            int maxIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] > unique[maxIndex]) {
                    maxIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[maxIndex];
            unique[maxIndex] = tempVal;
        }

        System.out.println("Unique count: " + size);
        System.out.print("Sorted array: ");
        for (int i = 0; i < size; i++) {
            System.out.print(unique[i] + " ");
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Input Handling & Initialization	1
2	Counting unique elements and creating a final unique array	3
3	Ascending sorting in even places	2.5
4	Descending sorting in an odd place	2.5
5	Prints the unique count and sorted array	1

SET - [B]

SOLUTION

```
public class SetB {
    public static void main(String[] args) {

        int[] scores = {4, 2, 6, 1, 4, 1};
        int[] unique = new int[scores.length];
        int size = 0;

        for (int i = 0; i < scores.length; i++) {
            boolean isDuplicate = false;
            for (int j = 0; j < size; j++) {
                if (scores[i] == unique[j]) {
                    isDuplicate = true;
                    break;
                }
            }
            if (isDuplicate==false) {
                unique[size] = scores[i];
                size++;
            }
        }

        for (int i = 0; i < size; i += 2) {
            int maxIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] > unique[maxIndex]) {
                    maxIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[maxIndex];
            unique[maxIndex] = tempVal;
        }
        for (int i = 1; i < size; i += 2) {
            int minIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] < unique[minIndex]) {
                    minIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[minIndex];
            unique[minIndex] = tempVal;
        }

        System.out.println("Unique count: " + size);
        System.out.print("Sorted array: ");
        for (int i = 0; i < size; i++) {
            System.out.print(unique[i] + " ");
        }
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Input Handling & Initialization	1
2	Counting unique elements and creating final unique array	3
3	Ascending sorting in even place	2.5
4	Descending sorting in odd place	2.5
5	Prints the unique count and sorted array	1